

Федеральное государственное автономное образовательное учреждение высшего
образования



**«Московский государственный технический университет
имени Н.Э. Баумана»**

(национальный исследовательский университет)

(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

О т ч е т

по лабораторной работе № 1

**Название лабораторной работы: Программирование с использованием
библиотеки QT.**

Дисциплина: Алгоритмизация и программирование

Студент гр. ИУ6-13Б _____ **С.М. Соболев**
(Подпись, дата) (И.О. Фамилия)

Преподаватель _____ **О.А. Веселовская**
(Подпись, дата) (И.О. Фамилия)

Москва, 2025

Часть 1. Создание простого приложения.

Цель работы: освоить фундаментальные принципы конструирования приложений с графическим интерфейсом (GUI) на базе фреймворка Qt.

Ход работы: был сгенерирован стартовый проект на C++ под управлением qmake. В редакторе кода реализована логика для изучения механизмов размещения элементов.

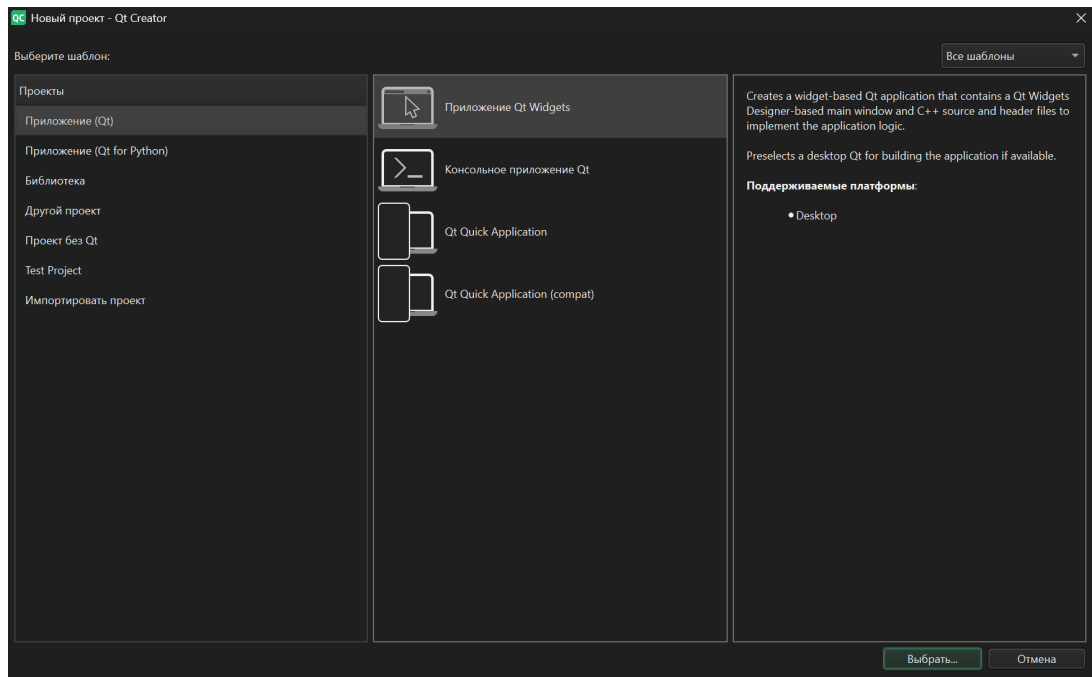


Рисунок 1 - Создание проекта

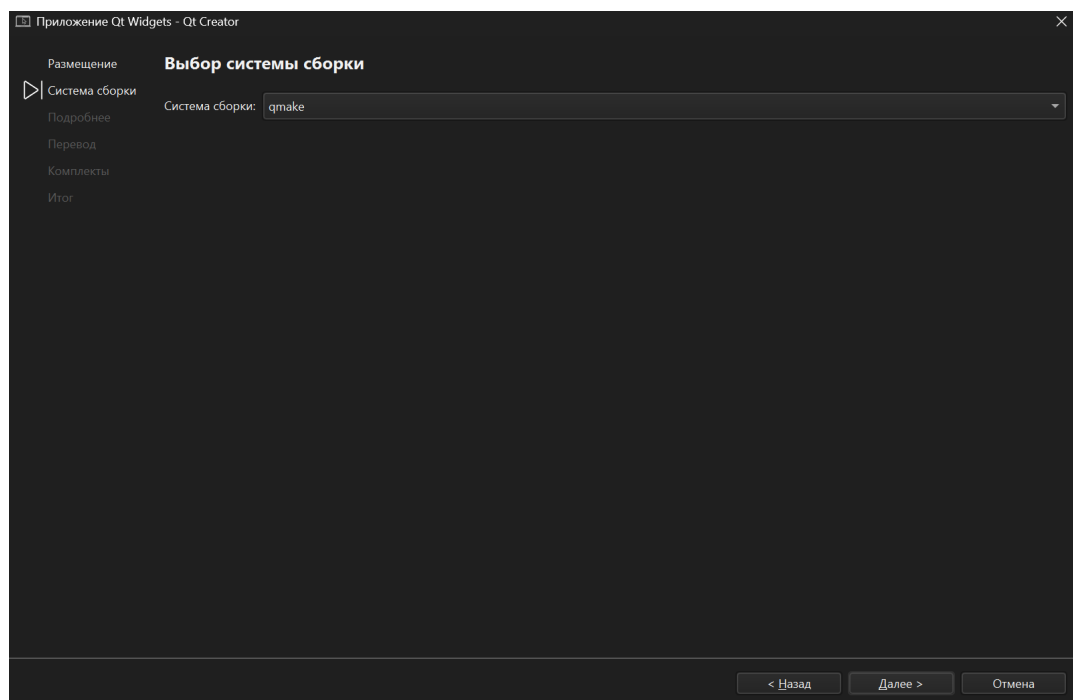


Рисунок 2 – выбор систему сборки

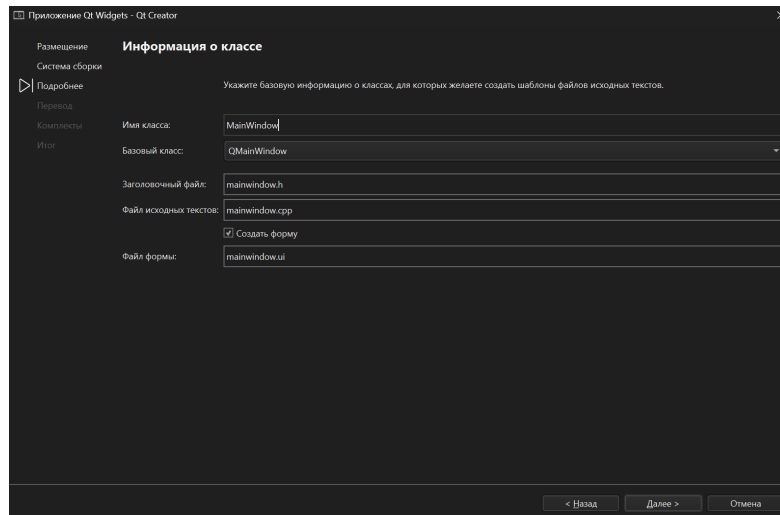


Рисунок 3 – Настройка проекта

В редактор кода была введена программа. Затем QHBoxLayout был изменен на QVBoxLayout.

```
#include <QApplication>
#include <QHBoxLayout>
#include <QVBoxLayout>
#include <QSlider>
#include <QSpinBox>
#include <QPushButton>
#include <QTextCodec>
#include <QVBoxLayout>
// Преобразуем входную последовательность символов в кодировку UNICODE
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    // Создаем главное окно
    QWidget *hbox = new QWidget();
    hbox->setWindowTitle("Введите Ваш Возраст");
    QSpinBox *spinBox = new QSpinBox( hbox );
    QSlider *slider = new QSlider(Qt::Horizontal, hbox );
    spinBox->setRange(0, 130);
    slider->setRange(0, 130);
    spinBox->setValue(35);
    QPushButton * btn = new QPushButton("Завершение", hbox );
    //*****
    QVBoxLayout *layout = new QVBoxLayout; // выравнивание по горизонтали
    //QVBoxLayout *layout = new QVBoxLayout; // выравнивание по вертикали

    hbox->setLayout(layout); // связываем layout с hbox
    // устанавливаем порядок следования элементов
    layout->addWidget(spinBox);
    layout->addWidget(slider);
    layout->addWidget(btn);
    //*****
    // связываем сигнал изменения spinBox со слотом slider
    QObject::connect(spinBox, SIGNAL(valueChanged(int)),
        | | | | slider, SLOT(setValue(int)));
    // связываем сигнал изменения slider со слотом spinBox
    QObject::connect(slider, SIGNAL(valueChanged(int)),
        | | | | spinBox, SLOT(setValue(int)));
    // связываем сигнал нажатия btn со слотом close главного окна
    QObject::connect(btn, SIGNAL(clicked(bool)),
        | | | | hbox, SLOT(close()));
    hbox->show(); // отображаем окно
    return app.exec(); // запускаем цикл обработки сообщений
}
```

Рисунок 4 – Код программы

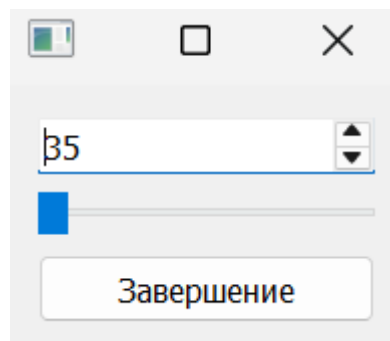


Рисунок 5 – Итоговая программа

Вывод: в результате получено понимание архитектуры библиотеки Qt, а также приобретены начальные навыки управления компоновкой виджетов, компиляции проекта и обработки пользовательского ввода.

Часть 2. Создание простого приложения в QtDesigner.

Цель работы: получить навыки проектирования интерфейсов через визуальный редактор Qt Designer и закрепить понимание механизма взаимодействия объектов.

Ход работы: в качестве основы была выбрана пустая диалоговая форма, имя объекта изменено, а для улучшения визуальной структуры макета добавлены разделители. Настроена синхронизация значений между элементами управления. Дополнительно реализовано всплывающее окно подтверждения выхода и применен компонент QSplitter для разделения области окна.

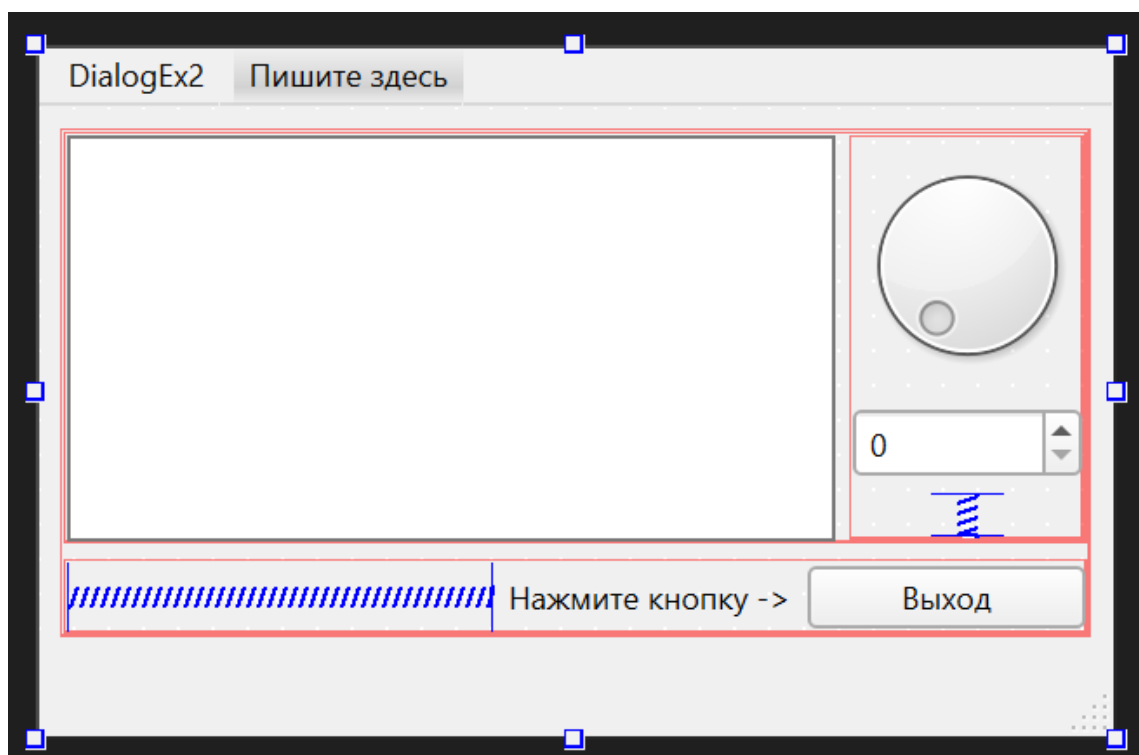


Рисунок 6 – Интерфейс созданный в QT Designer

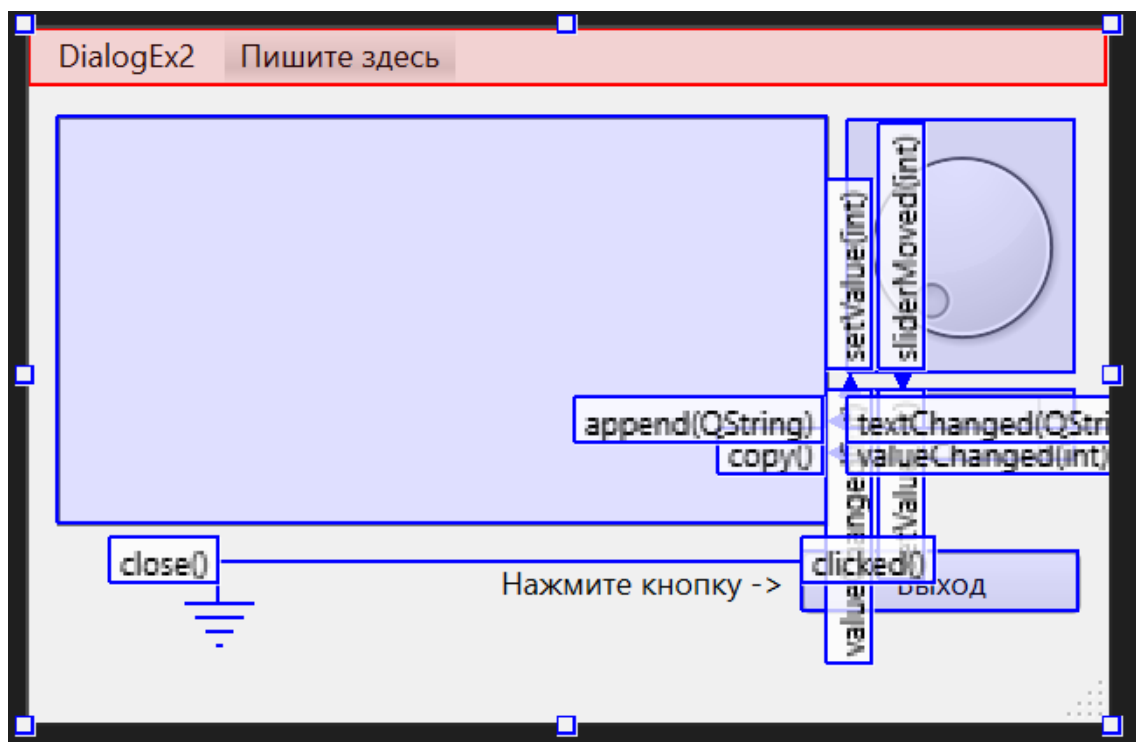


Рисунок 7 – Схема слотов

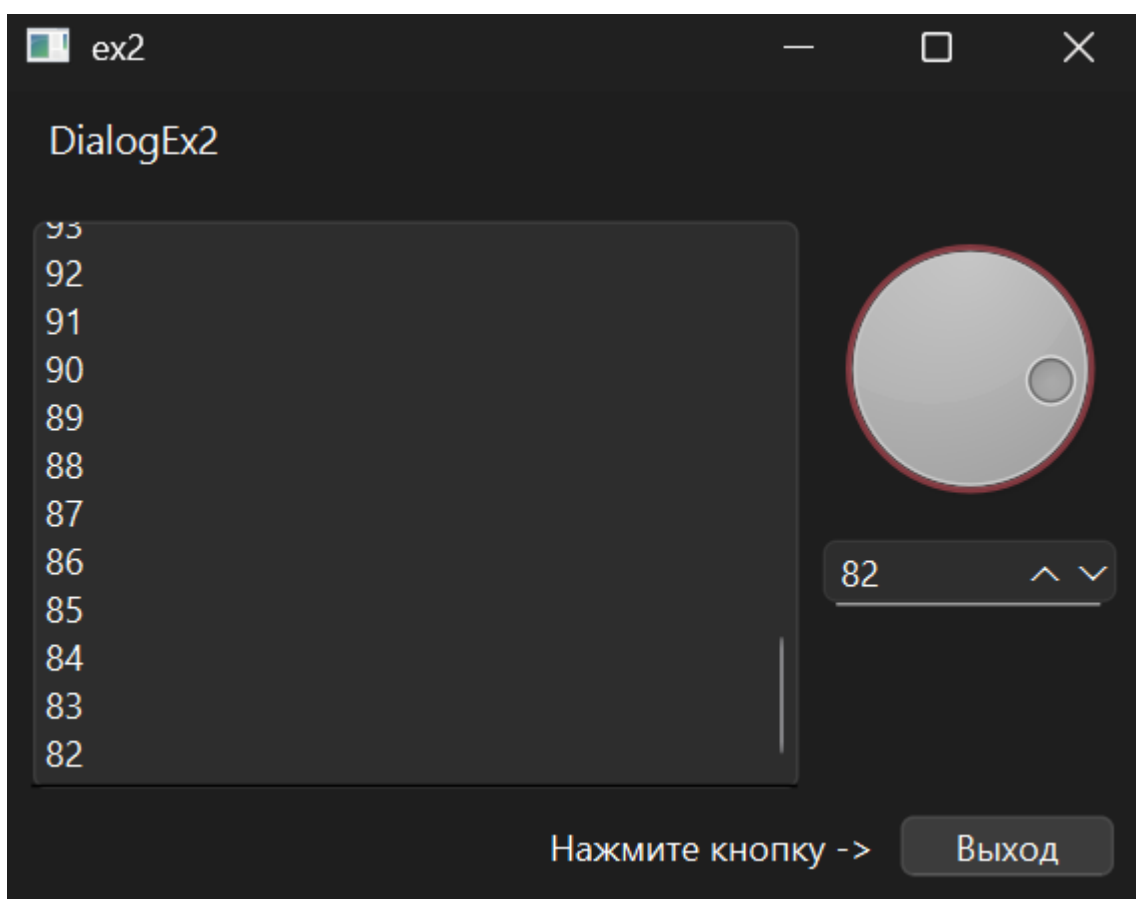


Рисунок 8 – Пример работы приложения

С помощью QSplitter (Рисунок 9) мы расположили рядом два одинаковых виджета (Рисунок 10).

```
1  #include "dialogex2.h"
2  #include <QApplication>
3  #include <QSplitter>
4  int main(int argc, char *argv[])
5  {
6      QApplication app(argc, argv);
7      QSplitter * splitter = new QSplitter(Qt::Vertical);
8      DialogEx2 * dialog1 = new DialogEx2();
9      DialogEx2 * dialog2 = new DialogEx2();
10     splitter->addWidget( dialog1 );
11     splitter->addWidget( dialog2 );
12     splitter->show();
13     return app.exec();
14 }
15
```

Рисунок 9 – Код программы с QSplitter

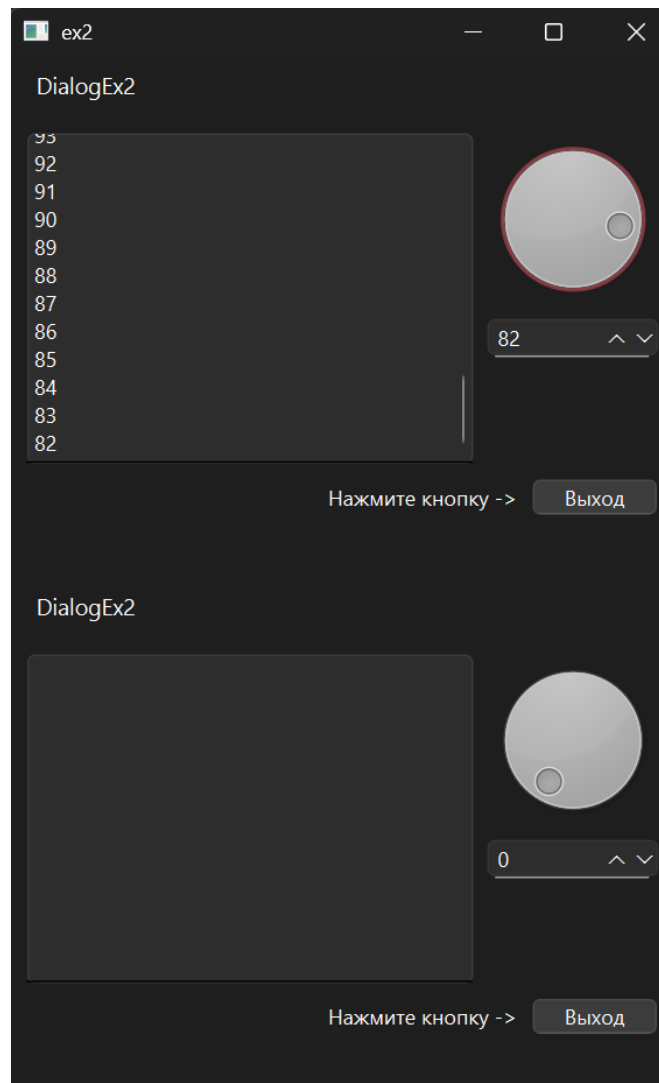


Рисунок 10 – Итоговое приложение

Вывод: сформировано представление о методах визуальной разработки форм в Qt Designer и практическом применении слотов.

Часть 3. Разработка калькулятора.

Цель работы: спроектировать программный калькулятор и нарастить его возможности за счет внедрения сложных математических операций.

Ход работы: сначала был сверстан стандартный интерфейс калькулятора. Затем функционал программы был существенно расширен: добавлены кнопки для новых вычислений и прописана логика их выполнения, включая необходимые проверки корректности данных. Для организации кнопок использовалось вертикальное выравнивание.



Рисунок 11 – Начальный дизайн приложения

Были добавлены в массив-вектор новые операции.

```
_btnDescr.push_back( BtnDescr("POW", POW) );  
_btnDescr.push_back( BtnDescr("LOG", LOG) );  
_btnDescr.push_back( BtnDescr("SIN", SIN) );  
_btnDescr.push_back( BtnDescr("COS", COS) );
```

Рисунок 12 – Добавленный код

```

case SIN:{
    setNumEdit( sin(getNumEdit()) ); break;
};
case COS:{
    setNumEdit( cos(getNumEdit()) ); break;
};

```

Рисунок 13 – Реализация унарных функций

```

case POW:{
    m_Val = pow(m_Val, num);
    break;
}
case LOG:{
    if (m_Val > 0 && m_Val != 1 && num > 0) m_Val = log(num) / log(m_Val);
    else m_Val = 0;
    break;
}

```

Рисунок 14 – Реализация бинарных функций

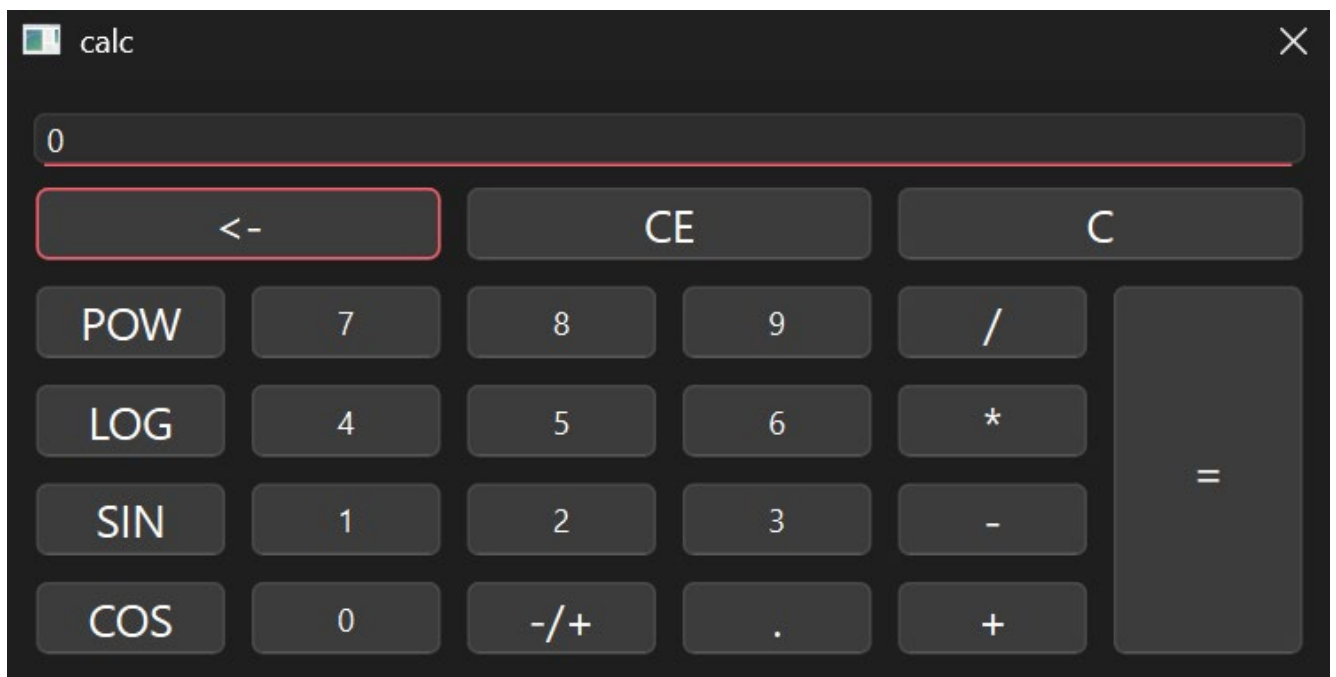


Рисунок 15 – Итоговый дизайн калькулятора

Вывод: успешно создано приложение-калькулятор на C++ с использованием Qt. Программа была модернизирована путем добавления новых операторов и изменения интерфейса.

Часть 4. Простейшие элементы ввода-вывода.

Цель работы: написать утилиту без использования графического конструктора, которая умеет трансформировать регистр текста по нажатию управляющей кнопки.

Ход работы: был вручную описан класс Dialog, содержащий как визуальную структуру, так и программную логику. Реализован алгоритм, при котором введенная строка меняет регистр при клике, а также предусмотрена реакция на завершение редактирования поля ввода. Интерфейс выстроен вертикально с помощью кода.

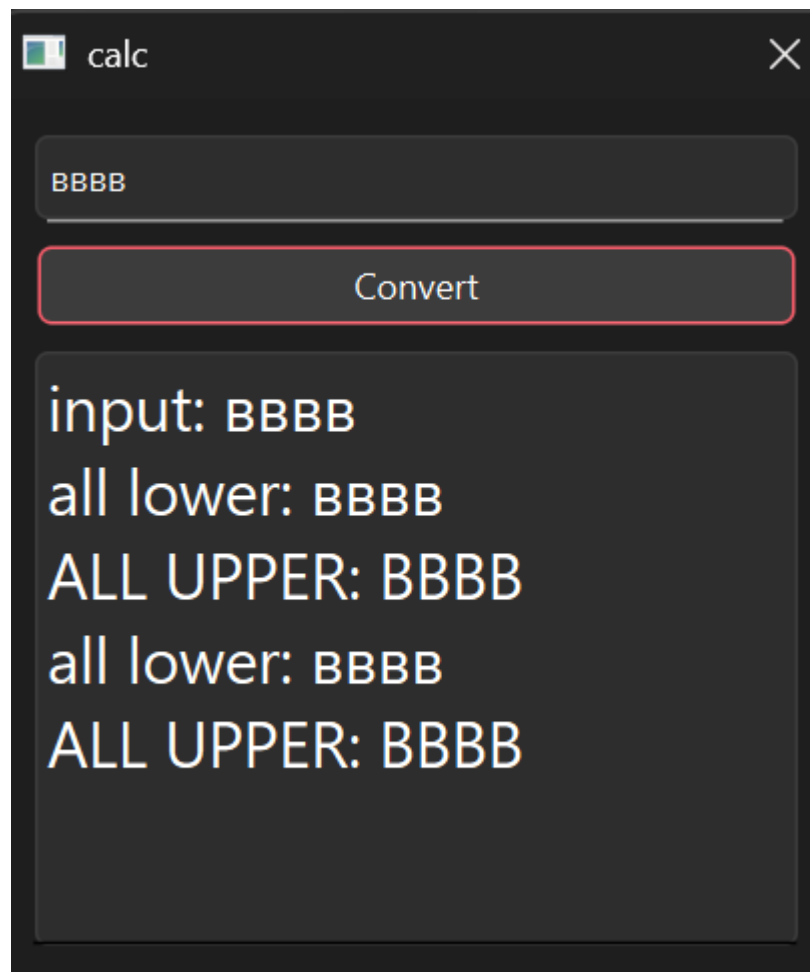


Рисунок 16 – Дизайн программы

```

1  #ifndef DIALOG_H
2  #define DIALOG_H
3  #include <QDialog>
4  #include <QLineEdit>
5  #include <QTextEdit>
6  /// Класс, реализующий калькулятор
7  class Dialog: public QDialog
8  {
9      Q_OBJECT
10 public:
11     Dialog( QWidget * parent = 0);
12     virtual ~Dialog(){};
13 protected:
14     QLineEdit *LineEdit1;
15     QTextEdit *LineEdit2;
16
17 private slots:
18     void convert();
19     void input();
20 };
21 #endif

```

Рисунок 17 – Заголовочный файл

```

Dialog::Dialog(QWidget *parent): QDialog(parent)
{
    LineEdit1 = new QLineEdit();
    LineEdit2 = new QTextEdit();
    QVBoxLayout *mainLayout = new QVBoxLayout();
    QPushButton *btn = new QPushButton("Convert");

    QFont font = LineEdit2->font();
    font.setPointSize(16); // Размер шрифта 16
    LineEdit2->setFont(font);
    LineEdit2->setMinimumHeight(200);
    LineEdit1->setMinimumHeight(30);
    btn->setMinimumHeight(30);

    LineEdit2->setReadOnly(true);
    mainLayout->addWidget(LineEdit1);
    mainLayout->addWidget(btn);
    mainLayout->addWidget(LineEdit2);
    setLayout(mainLayout);

    connect(btn, SIGNAL(clicked()), this, SLOT(convert()));
    connect(LineEdit1, SIGNAL(editingFinished()), this, SLOT(input()));
}

```

Рисунок 18 – Настройка интерфейса

```

void Dialog::convert(){
    static bool flag = 0;
    QString s = LineEdit1->text();
    if (flag) s = "ALL UPPER: " + s.toUpper();
    else s = "all lower: " + s.toLower();
    flag = !flag;
    LineEdit2->append(s);
}

void Dialog::input(){
    LineEdit2->setText("input: " + LineEdit1->text());
}

```

Рисунок 19 – Логика программы

```

#include "dialog.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    Dialog *w = new Dialog();
    w->show();
    return app.exec();
}

```

Рисунок 20 – main.cpp

Вывод: задача по манипуляции со строковыми данными выполнена. Разработано приложение, полностью отвечающее требованиям по функционалу и компоновке элементов.